

Факторизованная грамматика для продвинутого лямбда-интерпретатора

Условие

Требуется построить факторизованную однозначную грамматику без ε -переходов для языка лямбда-выражений с именованными определениями вида `let`. Поддерживаются:

- переменные;
- аппликация;
- лямбда-абстракция;
- скобки;
- несколько параметров у лямбда-абстракции;
- именованные определения.

Пустые строки и пробельные символы, не влияющие на структуру выражения, отбрасываются на этапе лексического анализа. Конец непустой строки представляется токеном `NL`. Конец входа представляется токеном `EOF`.

Терминалы

Терминалы задаются следующими правилами:

```
LET      ::= "let"
LAM      ::= "\"
DOT      ::= "."
EQ       ::= "="
LPAREN   ::= "("
RPAREN   ::= ")"
NL       ::= конец непустой строки
EOF      ::= конец входа
```

Терминал `IDENT` обозначает идентификатор. Ключевое слово `let` не считается идентификатором, поэтому его нельзя использовать в качестве имени переменной или имени определения.

Грамматика

Program ::= EOF
| Lines EOF

Lines ::= Line
| Line NL Lines

Line ::= Definition
| Expr

Definition ::= LET IDENT EQ Expr

Expr ::= Lambda
| Application

Lambda ::= LAM Params DOT Expr

Params ::= IDENT
| IDENT Params

Application ::= Atom
| Atom Application
| Atom Lambda

Atom ::= IDENT
| LPAREN Expr RPAREN

Свойства грамматики

- В грамматике отсутствуют ε -продукции.
- Грамматика не содержит левой рекурсии.
- Пустая программа задаётся правилом `Program ::= EOF`. Это не является ε -продукцией, так как `EOF` считается терминалом.
- Именованное определение начинается с ключевого слова `let`, поэтому оно однозначно отличается от выражения.
- Лямбда-абстракция однозначно распознаётся по начальному символу `\`.
- Аппликация задаётся как непустая последовательность применений.
- Аппликация может оканчиваться лямбда-абстракцией без скобок.

Последовательность атомов

`a1 a2 ... an`

при построении абстрактного синтаксического дерева интерпретируется как левоассоциативная аппликация:

$((a1\ a2)\ a3)\ \dots\ an)$

Также аппликация может оканчиваться лямбда-абстракцией без скобок. Например,

$\backslash x\ y.\ y\ x\ \backslash x.x$

понимается как

$\backslash x\ y.\ ((y\ x)\ (\backslash x.x))$

Лямбда-абстракция с несколькими параметрами

$\backslash x\ y\ z.M$

интерпретируется как

$\backslash x.\ \backslash y.\ \backslash z.M$

Пример вывода 1

Выведем выражение:

$\backslash x\ y.x$

Expr

=> Lambda

=> LAM Params DOT Expr

=> LAM IDENT Params DOT Expr

=> LAM IDENT IDENT DOT Expr

=> LAM IDENT IDENT DOT Application

=> LAM IDENT IDENT DOT Atom

=> LAM IDENT IDENT DOT IDENT

Подставляя конкретные идентификаторы, получаем:

$\backslash x\ y.x$

Пример вывода 2

Выведем выражение:

S K K

Expr

=> Application

=> Atom Application

=> IDENT Application

=> IDENT Atom Application

=> IDENT IDENT Application

=> IDENT IDENT Atom

=> IDENT IDENT IDENT

Подставляя конкретные идентификаторы, получаем:

S K K

Пример вывода 3

Выведем программу с именованным определением:

let I = \x.x

I y

Program

=> Lines EOF

=> Line NL Lines EOF

=> Definition NL Lines EOF

=> LET IDENT EQ Expr NL Lines EOF

=> LET IDENT EQ Lambda NL Lines EOF

=> LET IDENT EQ LAM Params DOT Expr NL Lines EOF

=> LET IDENT EQ LAM IDENT DOT Expr NL Lines EOF

=> LET IDENT EQ LAM IDENT DOT Application NL Lines EOF

=> LET IDENT EQ LAM IDENT DOT Atom NL Lines EOF

=> LET IDENT EQ LAM IDENT DOT IDENT NL Lines EOF

=> LET IDENT EQ LAM IDENT DOT IDENT NL Line EOF

=> LET IDENT EQ LAM IDENT DOT IDENT NL Expr EOF

=> LET IDENT EQ LAM IDENT DOT IDENT NL Application EOF

=> LET IDENT EQ LAM IDENT DOT IDENT NL Atom Application EOF

=> LET IDENT EQ LAM IDENT DOT IDENT NL IDENT Application EOF

=> LET IDENT EQ LAM IDENT DOT IDENT NL IDENT Atom EOF

=> LET IDENT EQ LAM IDENT DOT IDENT NL IDENT IDENT EOF

Подставляя конкретные идентификаторы, получаем:

let I = \x.x

I y

Пример вывода 4

Выведем выражение, где аппликация оканчивается лямбда-абстракцией без скобок:

```
y x \x.x
```

```
Expr
=> Application
=> Atom Application
=> IDENT Application
=> IDENT Atom Lambda
=> IDENT IDENT Lambda
=> IDENT IDENT LAM Params DOT Expr
=> IDENT IDENT LAM IDENT DOT Expr
=> IDENT IDENT LAM IDENT DOT Application
=> IDENT IDENT LAM IDENT DOT Atom
=> IDENT IDENT LAM IDENT DOT IDENT
```

Подставляя конкретные идентификаторы, получаем:

```
y x \x.x
```

Это соответствует выражению:

```
((y x) (\x.x))
```

Пример программы

```
let S = \x y z.x z (y z)
let K = \x y.x
S K K
```

Эта программа является последовательностью строк. Первые две строки являются именованными определениями, а последняя строка является лямбда-выражением.

Также грамматика допускает определения не только в начале программы, например:

```
let I = \x.x
I y
let t = a b c
```

Это корректная программа, так как каждая непустая строка является либо именованным определением, либо лямбда-выражением.